<Name-of-Software-Application>

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 6

Document Revision History

Version	Date	Author	Comments
1.0	01/24/26	Tyrone Grant	Adding to executive summary, design requirements, constraints, domain model description, operating system evaluations, and recommendations.

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The client, The Gaming Room, is looking to have their existing mobile app *Draw It or Lose It* developed into a web-based app that can be run on various platforms. The ideal solution for the client involves using a coding framework that allows us to create applications for multiple platforms. This multi-platform coding framework may lead to some inconsistencies between platforms (web-based vs. Android) and limited access to platform-specific features in app.

Requirements

Nonfunctional Requirements:

1. Application should run on Windows, Linux, iOS, and Android operating systems.
2. Servers should support low latency communication between users to facilitate gameplay.
3. Application should share a similar user interface between platforms to ensure users have a standardized experience regardless of the platform used to run the application.

Functional Requirements:

1. A game will have the ability to have one or more teams involved.
2.

Design Constraints

1. The application must be developed using a coding framework for multi-platform applications.
2. High performance servers must be used for low latency communication amongst users.
3. UI & UX must be standardized.
4. The application must have a real-time check for existing team and player names.

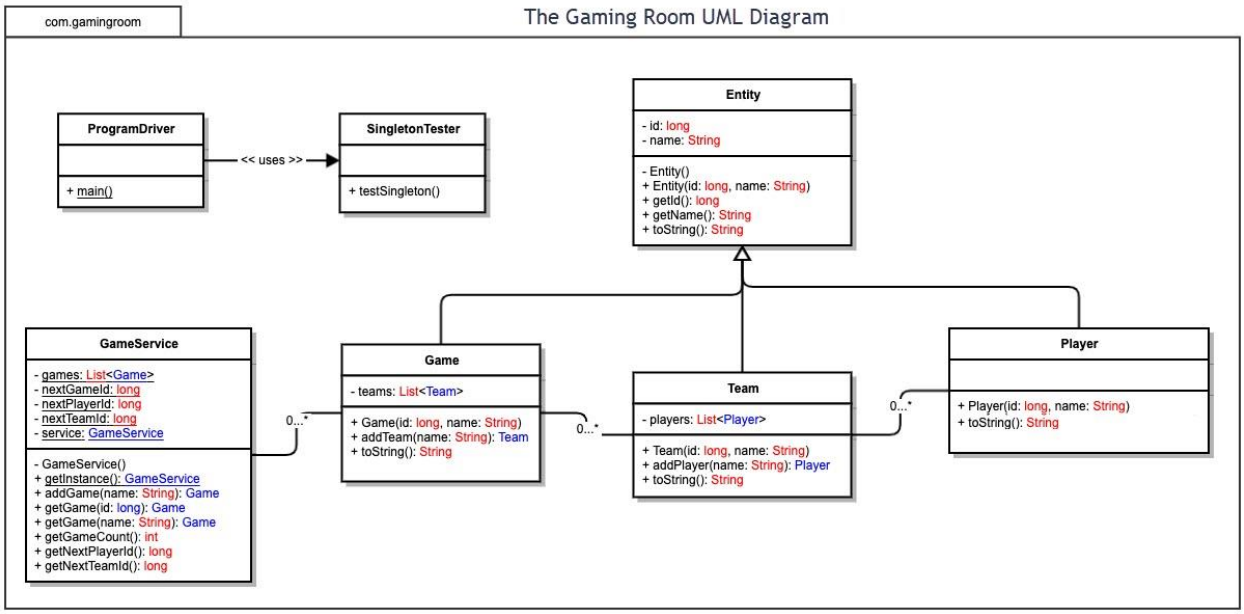
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

<Describe the UML class diagram provided below. Explain how the classes relate to each other. Identify any object-oriented programming principles that are demonstrated in the diagram and how they are used to fulfill the software requirements efficiently.>
ProgramDriver: Houses the main method which executes the program. Uses the SingletonTester class.
SingletonTester: Contains the Singleton method. Class is used to ensure that one instance of the game exists in memory at a time.
Entity: Superclass that contains a constructor, attributes, and methods. The **Game**, **Team**, and **Player** classes are its subclasses. The class supports the principle of encapsulation by preventing its attributes from being altered directly by users.
GameService: Houses a list, attributes, constructors, and methods. The class handles the implementation of the **Game** subclass and covers most of the software requirements by creating a list of

teams, allowing players to be added to a team list, and checking for existing team and player names. The class supports the principle of abstraction by hiding information that isn't required for the user to use the class.
Game: Contains a list of teams, a constructor, and methods. The class supports the principles of polymorphism and inheritance because it's a subclass that has features that are different from the superclass.
Team: *Description is similar to the **Game** class*
Player: *Description is similar to the **Game** class*



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices

Server Side	**Characteristics:** - Stable - Strong security **Advantages:** - Reliable for development - Good performance **Weaknesses:** - Expensive hardware - Less common for large-scale hosting	**Characteristics:** - Open-source - Highly customizable - Widely used for servers **Advantages:** - Excellent performance - Strong security **Weaknesses:** - Higher learning curve for beginners.	**Characteristics:** - User-friendly - Strong organizational support via Microsoft Advantages: - Familiar UI Weaknesses: - Licensing costs - Heavier resource usage - Less common for web servers	**Characteristics:** - Limited server hosting capability **Advantages:** - Useful for testing **Weaknesses:** - Not suitable for primary web hosting due to power and scalability constraints
Client Side	**Cost:** - High (due to hardware costs and licensing) **Time:** - Coding framework for multi-platform applications reduces development time **Expertise:** **-** MacOS and safari environments	**Cost:** - Low (open-source) **Time:** - Coding framework for multi-platform applications reduces development time **Expertise:** **-** Command-line knowledge - Server expertise - Cross-browser testing	**Cost:** - Medium (due to licensing costs) **Time:** - Coding framework for multi-platform applications reduces development time **Expertise:** **-** Windows and Microsoft Edge environments	**Cost:** - Medium (due App Store/Google Play Store fees) **Time:** - Coding framework for multi-platform applications reduces development time **Expertise:** **- i**OS and Android - Mobile UX design
Development Tools	**Languages:** - JavaScript - Python - Java - Swift **Tools:** - VS Code - Xcode - Homebrew - Docker	**Languages:** - JavaScript - Python - Java - PHP - Go **Tools:** - VS Code - Docker - Vim - Git - Apache	**Languages:** - JavaScript - Python - Java - C# **Tools:** - VS Code - Docker - Visual Studio - IIS	**Languages:** - JavaScript - Swift - Kotlin - Dart **Tools:** - Xcode - Android Studio - Flutter - React Native

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**:
   As it pertains to an operating platform, I would recommend using Windows because it supports cross-platform clients, integrates well with other environments, and has a familiar interface for administrators.

2. **Operating Systems Architectures**:
   The operating system architecture is known to support multiplayer gaming and has enterprise-grade reliability.

3. **Storage Management**:
   The appropriate storage management system would be cloud-based and database storage.

4. **Memory Management**:
   Windows utilizes virtual memory, dynamic memory allocation, and process isolation to manage memory. These techniques would provide players with stable gameplay while multiple users are playing simultaneously and efficient RAM utilization for smooth gameplay.

5. **Distributed Systems and Networks**:
   A server would host Draw It or Lose It and clients would connect to the server using various platforms (Mac, Windows, Android, etc.). Communication between clients and the server that hosts the game would take place via HTTPS and traffic would be distributed accordingly.

6. **Security**:
   Offering a safeguarded environment for users to game safely and securely is of the utmost importance. Draw It or Lose It requires features such as secure login and authentication, data encryption, and role-based permissions to protect its users. Windows provides security features like Windows Defender and firewall protection to protect devices from malware. Alongside those defenses, it gives users access control and permissions restrictions to control access to information and processes on device and protects against unauthorized network access in addition to encrypting network traffic between clients/servers. In consideration of those broad defense features, Windows would offer users a robustly secure platform to play Draw It or Lose It.